UNIVERSIDAD ICESI · FACULTAD DE INGENIERÍA · DEPARTAMENTO DE TIC

Discrete Structures III (CyED III, Código 09772) · October 2026

# ResumeLens: Formal Language-Based Resume Screening

## Automated Candidate Qualification Auditing Using Transducers, Automata, and Context-Free Grammars

**Nicolás Santiago Buelvas González**
Stages 1 & 2: Regex Extraction & FST Normalization
**Lead Formal Modeling & Normalization Pipeline**

**Daniel Alexander Nastul Flórez**
Stages 3 & 4: Profile Automata & textX Grammar
**DFA State Modeling & Abstract Syntax Trees**

**Albert Valencia Escobar**
Stage 5: Visualization & Empirical Verification
**Advisor: Prof. Andrés Aristizábal (Grupo 3)**

## 1. Problem Statement & Motivation

RESEARCH BACKGROUND

- **Lexical Fragmentation:**
  The exact same technical competency occurs in diverse textual surface forms (e.g. PostgreSQL, Postgres, postgres, psql). Pure keyword search fails to unify semantic equivalence across heterogeneous applicant curricula.
- **Order Insensitivity in Naive Systems:**
  Traditional ATS bags-of-words ignore structural prerequisites. A senior stack requires a foundational language, frontend UI framework, backend server runtime, and persistence layer; bag matching accepts incomplete or scrambled submissions.
- **The Black-Box Interpretability Deficit:**
  Deep learning classifiers and generative LLMs produce opaque, probabilistic candidate ratings vulnerable to hallucinations, drift, and lack of reproducible audit trails required by employment compliance standards.

**Formal Thesis:**
Formal language theory (Chomsky Hierarchy: Regular, FST, DFA, CFG) yields a 100% deterministic, mathematically auditable pipeline that guarantees zero hallucination and complete decision traceability.

## 2. Formal Methodology & 5-Stage Architecture

CHOMSKY HIERARCHY

**STAGE 1: EXTRACTION** — Regular Expressions
Extracts email, phone, LinkedIn/GitHub, and raw technical surface tokens. `L(R) subset Sigma_text*`

**STAGE 2: NORMALIZATION** — Finite-State Transducers
Normalizes surface variants to canonical tokens & sorts by role hierarchy. `M = (Q, Sigma, Gamma, delta, omega, q0, F)`

**STAGE 3: RECOGNITION** — Deterministic Automata
Validates candidate skill sequences against 4 strict role stack DFAs. `M = (Q, Sigma, delta, q0, F)`

**STAGE 4: SPECIFICATION** — textX CFG Grammar
Parses candidates into strongly typed Abstract Syntax Trees (AST). `G = (V_N, V_T, P, S)`

**STAGE 5: VERIFICATION** — Report Generation
Produces inspectable HTML candidate scorecards, Markdown, and DSL. `Audit Trail Artifacts`

## 3. Normalization Transducers & Canonical Ordering

STAGE 2 FST

**Formal 7-Tuple Definition:** `M = ( Q, Sigma, Gamma, delta, omega, q0, F )`

**Input Alphabet (Sigma):** 47 symbols occurring in spelling variants: ASCII letters, digits, '.', '-', and ' '.
**Output Alphabet (Gamma):** 34 canonical tokens { JAVASCRIPT, REACT, POSTGRESQL, SCIKIT_LEARN, ... }
**Transition Relation (delta):** Q x Sigma -> 2^Q (Nondeterministic prefix tree sharing common root state q0).
**Emission Function (omega):** omega(q, c) = epsilon for prefixes; omega(q_end, c) = K in Gamma upon reaching final state.
**Transducer Engines:** M_SE: 438 states, 66 final states (Software) | M_AI: 367 states, 55 final states (Data/ML)

**Transducer Emission Mechanics Sub-Table:**

| Source State | Input (c) | Target State | Emission (omega) |
|---|---|---|---|
| q0 | 'J' | q_JS1 | epsilon (silent prefix) |
| q_JS1 | 'S' | q_JS2 in F | JAVASCRIPT (canonical output) |
| q_React6 | 'j' | q_React7 | epsilon (silent prefix) |
| q_React7 | 's' | q_React8 in F | REACT (canonical output) |

**Order-Invariance & Categorical Sorting Function:**

`Sort(S, P) = order by ( Priority(Category(s, P)), s )  for all s in (S intersect Sigma_P)`

1. Discards tokens outside profile alphabet Sigma_P. 2. Orders tokens by architectural priority tiers.
3. Guarantees deterministic sequence execution in O(N log N) without DFA state explosion.

## 4. Context-Free Grammar (CFG) & textX DSL

STAGE 4 DSL

**Formal 4-Tuple Model:** `G = ( V_N, V_T, P, S )`

**Non-Terminals (V_N):** { ResumeModel, Candidate, ContactInfo, Skills, Evaluation, ProfileStatus }
**Terminals (V_T):** { 'candidate', 'skills', 'evaluation', 'ACCEPTED', 'REJECTED', STRING, INT }
**Start Symbol (S):** ResumeModel (Top-level root node of the parsed Abstract Syntax Tree)

```
// Canonical textX Serialized Candidate Model Output
resume LinaMarquez {
  candidate { name: "Lina Marquez"  email: "lina.marquez@example.com" }
  skills { canonical: [JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT] }
  evaluation {
    profile "Full Stack Developer": ACCEPTED
    profile "Backend Developer": ACCEPTED
    profile "Machine Learning Engineer": REJECTED
  }
}
```

## 5. Deterministic Finite Automata (DFA) for Profiles

STAGE 3 DFA

**Formal 5-Tuple Specification:** `M_profile = ( Q, Sigma, delta, q0, F )`
Deterministic execution: for every state q and symbol a, |delta(q, a)| <= 1. No epsilon-transitions. Missing transition = rejection.

| Profile Target | \|Q\| | Start | Accepting States (F) | Classification |
|---|---|---|---|---|
| Full Stack Developer | 6 | q0_start | { q5_final } | Deterministic FA |
| Machine Learning Eng. | 6 | q0_start | { q5_final } | Deterministic FA |
| Backend Developer | 4 | q0_start | { q3_tools } | Deterministic FA |
| Data Engineer | 5 | q0_start | { q3_proc, q4_final } | Deterministic FA |

**Automata Transition Tier Hierarchies:**

**Full Stack Developer Tier Sequence (Accepted: Lina Marquez):**

```
q0_start  --[JAVASCRIPT]-->  q1_lang  --[REACT]-->  q2_fe  --[NODE_JS]-->
q3_be     --[POSTGRESQL]-->  q4_db    --[GIT]-->    q5_final in F  ==> ACCEPTED
```

**Machine Learning Engineer Trace Fault Analysis (Rejected: Dario Pineda):**

```
q0_start  --[PYTHON]-->  q1_lang  --[PANDAS]-->  q2_prep (Data prep verified)
q2_prep   --[POSTGRESQL]--> UNDEFINED TRANSITION delta(q2_prep, POSTGRESQL) = 0
```

Cause: Missing ML framework (Scikit-Learn/PyTorch). Automaton terminates in immediate formal rejection.

## 6. Empirical Results & Verification Rigor

SYNTHETIC DATASET

**Synthetic Resumes Dataset Classification Observations:**

| Resume File | Candidate | Target Profile | Observed Accepted Profiles |
|---|---|---|---|
| fullstack_valid.txt | Lina Marquez | Full Stack Dev | Full Stack Developer, Backend Developer |
| fullstack_invalid.txt | Mateo Rios | Full Stack Dev | Backend Developer (Full Stack rejected) |
| backend_sample.txt | Sara Beltran | Backend Dev | Backend Developer |
| ml_engineer_valid.txt | Celeste Arboleda | ML Engineer | ML Engineer, Backend Developer |
| ml_engineer_invalid.txt | Dario Pineda | ML Engineer | Backend Developer, Data Eng (ML rejected) |
| data_engineer_sample.txt | Rafaela Quintero | Data Engineer | Data Engineer |

**Empirical Finding — Profile Overlap:**
Full Stack and ML candidates frequently satisfy the Backend Developer automaton. This reflects real-world skill subsetting (Full Stack includes backend skills) rather than a classification defect.

**Test Suite Rigor: 100 of 100 Tests Passing (Execution: 4.24s)**

**100**
100% TESTS PASS

**test_e2e_resumes.py (37 tests):** End-to-end dataset screening & output idempotence check.
**test_extraction.py (15 tests):** Regex pattern boundaries for contact info and technical skills.
**test_normalization.py (10 tests):** FST prefix transitions, canonical emissions, and alias mappings.
**test_recognition.py (9 tests):** DFA state transitions, accepted/rejected paths, order invariance.

## 7. Conclusions, Formal Limitations & Future Work

OPEN RESEARCH

**Scientific Appraisal & Scope of Evidence:**

- **Formal Acceptance != Job Competence:**
  An ACCEPTED status verifies exclusively that the candidate's qualification string satisfied the formal grammar and transition function. It does not measure coding proficiency, soft skills, or real-world productivity.
- **Scope of Evidence:**
  Tests exercise synthetic curricula. Real-world resumes contain OCR errors, non-standard visual columns, and unstructured conversational narratives.
- **Lexical Horizon:**
  Currently bounded to the 47-symbol input alphabet and 34 configured canonical skills.

**Future Theoretical Extensions:**

1. **Type-1 Grammars (CSG):** Model cross-field constraints (e.g. binding years of experience to skills).
2. **Multilingual Transducers:** Expand the input alphabet to support Spanish, Portuguese, and international syntax.
3. **Weighted Automata (WFA):** Incorporate probabilistic edge weights for ranking while preserving determinism.